

DCCA201

Computer Architecture

Hyper-Structured Exam Master Notes

II Semester BCA — NEP Scheme | June/July 2025 Paper Analysis
Max Marks: 60 | Time: 2½ Hours | All Sections Covered

SECTION A	SECTION B	SECTION C
2 Marks × 4 = 8	5 Marks × 4 = 20	8 Marks × 4 = 32
Answer ANY 4 of 6	Answer ANY 4 of 6	Answer ANY 4 of 6

TABLE OF CONTENTS

■ SECTION A — 2 Mark Questions

18 topics, all definitions & short answers

1. Computer Architecture — Definition + components
2. Bit & Byte — Units + examples
3. Op-code & Operand — Instruction format
4. Micro Operation — Types + examples
5. Cache Memory — Levels + working
6. Peripheral Devices — 4 examples
7. Gray Code Conversion — Binary → Gray method
8. Number Conversions — Binary, Octal, Decimal, Hex
9. De Morgan's Theorem — Both laws + proof
10. Flip-Flop — Types + definitions
11. SRAM vs DRAM — Comparison table
12. Computer Registers — All 8 registers
13. IC & Types — SSI/MSI/LSI/VLSI
14. Hit Ratio — Formula + meaning
15. MIMD — Flynn's taxonomy
16. NOR / NOT Gate — Symbol + truth table
17. Control Inputs — 3 register control signals
18. Multiprocessor Characteristics — Key traits

■ SECTION B — 5 Mark Questions

10 topics, medium depth

19. K-Map Simplification — 4-variable, SOP method
20. 4-to-1 Multiplexer — Circuit + truth table
21. Half Adder & Full Adder — Logic + truth tables
22. BUN & BSA Instructions — Branch instructions
23. CISC vs RISC — Detailed comparison
24. Associative Memory — CAM architecture
25. ILP & Limitations — Instruction-level parallelism
26. Binary Codes — Gray, BCD, ASCII
27. Arithmetic Micro-operations — ADD, SUB, INC, DEC
28. 3-to-8 Line Decoder — Truth table + logic

■ SECTION C — 8 Mark Questions

15 topics, full depth

29. SR Flip-Flop — NOR/NAND construction, truth table
30. DMA Controller — Block diagram, working

- 31. 3×8 Line Decoder — Full diagram + truth table
- 32. Instruction Cycle — All phases + flowchart
- 33. Addressing Modes — 5 modes with examples
- 34. Multithreaded Architecture — Fine/coarse-grained
- 35. Shift Registers — SIPO, PISO, parallel load
- 36. Register-Reference Instructions — All 12 operations
- 37. Cross-bar & Multi-stage Switch — Network topology
- 38. CPU Organization — Single/multiple bus
- 39. Full Adder (detailed) — Circuit + Boolean
- 40. De Morgan's Theorem (proof) — Set theory proof
- 41. CISC vs RISC (8-mark) — Extended comparison
- 42. I/O Interface Unit — Interface architecture
- 43. Multiprocessor vs Multicomputer — Architecture diff

SECTION A

Short Answer Questions

2 Marks Each | Answer
ANY 4 of 6

Q1. Define Computer Architecture

■ CA defines the functional behaviour of a computer system as seen by the programmer.

Computer Architecture describes the **logical structure and functional characteristics** of a computer, including the instruction set, data formats, addressing modes, and I/O mechanisms. It operates at the interface between hardware and software.

Core Components:

- **Instruction Set Architecture (ISA):** The set of instructions the CPU can execute (e.g., ADD, LOAD, JUMP).
- **Microarchitecture:** The physical implementation of the ISA — pipeline stages, cache hierarchy, branch prediction.
- **System Architecture:** Memory subsystem, I/O buses, interrupt handling, and multiprocessor interconnects.

Level	Focus	Example
ISA	Programmer-visible operations	x86, ARM, MIPS instruction sets
Microarchitecture	Hardware execution details	Pipeline depth, cache size
System Design	Component integration	Bus width, memory hierarchy
Logic Design	Gate-level implementation	ALU using NAND gates

[INSIGHT] Architecture is WHAT the computer does; Organisation is HOW it does it. This distinction is a frequent 2-mark trap question.

Q2. What is Bit and Byte?

■ *A bit is the atomic unit of digital information; a byte is the practical addressable unit.*

Bit (Binary Digit): The smallest unit of data in computing, representing one of two states — 0 (off/false) or 1 (on/true). All digital data is ultimately stored and processed as combinations of bits.

Byte: A group of **8 bits**. A byte can represent $2^8 = 256$ distinct values (0–255). It is the standard addressable unit of memory in most architectures.

Unit	Size	Range / Example
Bit	1 bit	0 or 1
Nibble	4 bits	0000 to 1111 (0–15)
Byte	8 bits	00000000 to 11111111 (0–255)
Kilobyte (KB)	1024 bytes	~1000 characters of text
Megabyte (MB)	1024 KB	~1 million characters
Gigabyte (GB)	1024 MB	~1 billion characters

[EXAMPLE] The letter 'A' is stored as the byte 01000001 (ASCII value 65). Each pixel in an image may use 3 bytes (RGB).

Q3. Define Op-code and Operand

■ *Every machine instruction is split into WHAT to do (opcode) and WHAT to do it on (operand).*

A machine instruction has two fundamental fields:

- **Op-code (Operation Code):** The portion of the instruction that specifies the operation to be performed. Examples: ADD (addition), SUB (subtraction), LOAD (fetch from memory), JUMP (branch). In a 16-bit instruction, the opcode may occupy the upper 4 bits, allowing $2^4 = 16$ distinct operations.
- **Operand:** The data value or address on which the operation acts. Can be an immediate value, a register reference, or a memory address.

Instruction Format Example (16-bit):

| 0010 | 0000 0000 0110 |

^OP ^--- Operand (memory address 6) ---^

ADD

[EXAM TRAP] The opcode does NOT contain data — it contains the operation identifier. The operand may itself be an address (indirect) or value (immediate). Know the difference.

Q4. Define Micro Operation

■ *Micro-operations are the atomic hardware-level actions that collectively implement one machine instruction.*

A **Micro-operation** is a fundamental, indivisible operation performed by the CPU's control unit during one clock cycle. A single machine instruction decomposes into a sequence of micro-operations executed on registers, the ALU, or memory buses.

Four Classes of Micro-operations:

Class	Operation	Example
Register Transfer	Copy data between registers	$R1 \leftarrow R2$
Arithmetic	Math on register contents	$R3 \leftarrow R1 + R2$
Logic	Bitwise operations	$R1 \leftarrow R1 \text{ AND } R2$
Shift	Shift register contents	$R1 \leftarrow \text{shl } R1$ (left shift)

[DEEP-DIVE] During FETCH phase: $PC \rightarrow \text{MAR}$ (transfer), $M[\text{MAR}] \rightarrow \text{MDR}$ (memory read), $\text{MDR} \rightarrow \text{IR}$ (register transfer), $PC \leftarrow PC+1$ (arithmetic). These are all micro-operations of the single FETCH instruction.

Q5. Define Cache Memory

■ *Cache memory eliminates the speed mismatch between fast CPU and slow main memory by storing frequently used data close to the processor.*

Cache memory is a **small, high-speed volatile memory** located between the CPU and main RAM. It exploits the **principle of locality** — the observation that programs tend to access the same memory locations repeatedly (temporal locality) or nearby locations (spatial locality).

Cache Level	Location	Size	Speed	Purpose
L1	Inside CPU core	32–256 KB	~1 ns	Fastest; instruction + data
L2	On CPU die	256 KB – 4 MB	~5 ns	Bridge L1 and L3
L3	Shared across cores	4 – 64 MB	~20 ns	Last cache before RAM

Hit Ratio Formula:

$\text{Hit Ratio (h)} = \text{Cache Hits} / (\text{Cache Hits} + \text{Cache Misses})$

$\text{Avg Access Time} = h \times T_c + (1-h) \times T_m$

where T_c = cache access time, T_m = main memory access time

[INSIGHT] A hit ratio of 0.9 means 90% of memory accesses are served from cache. Modern CPUs achieve >95% L1 hit rates on typical workloads.

Q6. Write Any Four Peripheral Devices

■ *Peripheral devices extend the core computing system with input, output, and storage capabilities.*

Device	Category	Function	Interface
Keyboard	Input	Converts keystrokes to ASCII codes sent to CPU	USB / PS2
Mouse	Input	Translates physical movement to pointer coordinates	USB / Bluetooth
Printer	Output	Renders digital data as physical hard-copy output	USB / LAN
Hard Disk Drive	Storage (I/O)	Persistent magnetic storage of programs and data	SATA / NVMe
Monitor	Output	Renders framebuffer pixel data as visible image	HDMI / VGA
Scanner	Input	Converts physical documents to digital bitmap images	USB

Q7. Convert Binary to Gray Code

■ *Gray code changes only ONE bit between consecutive values — critical for eliminating transition errors in digital systems.*

Conversion Rule: Binary → Gray

- MSB of Gray code = MSB of Binary (unchanged).
- Each subsequent Gray bit = XOR of current Binary bit with previous Binary bit.
- Formula: $G[i] = B[i] \text{ XOR } B[i-1]$

Worked Example: Convert $(11011)_2$ to Gray Code

Binary: 1 1 0 1 1

Step 1: 1 (MSB, copied directly)

Step 2: 1 XOR 1 = 0

Step 3: 1 XOR 0 = 1

Step 4: 0 XOR 1 = 1

Step 5: 1 XOR 1 = 0

Result: 1 0 1 1 0 (Gray Code)

Decimal	Binary	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100

[EXAM TRAP] Gray code is NOT BCD. Gray code has only 1-bit difference between adjacent values. BCD encodes each decimal digit in 4 binary bits independently.

Q8. Number System Conversions

■ **Master the conversion chain: Decimal $\leftrightarrow$ Binary $\leftrightarrow$ Octal $\leftrightarrow$ Hex — these appear every semester.**

Example: Convert $(642)_{10}$ to Binary

$$642 \div 2 = 321 \text{ R } 0$$

$$321 \div 2 = 160 \text{ R } 1$$

$$160 \div 2 = 80 \text{ R } 0$$

$$80 \div 2 = 40 \text{ R } 0$$

$$40 \div 2 = 20 \text{ R } 0$$

$$20 \div 2 = 10 \text{ R } 0$$

$$10 \div 2 = 5 \text{ R } 0$$

$$5 \div 2 = 2 \text{ R } 1$$

$$2 \div 2 = 1 \text{ R } 0$$

$$1 \div 2 = 0 \text{ R } 1$$

Read remainders bottom-up: $(1010000010)_2$

Example: Convert $(11011)_2$ to Gray Code $\rightarrow (10110)_{\text{Gray}}$

Example: Convert $(11011)_2$ to Octal

Group in 3s from right: 011 011

$011 = 3, 011 = 3 \rightarrow (33)_8$

Q9. State De Morgan's Theorem

■ **De Morgan's laws are the foundation of logic gate equivalence and Boolean simplification.**

First Law: $(A + B)' = A' \cdot B'$

The complement of a sum equals the product of the complements. A NOR gate is equivalent to an AND gate with inverted inputs.

Second Law: $(A \cdot B)' = A' + B'$

The complement of a product equals the sum of the complements. A NAND gate is equivalent to an OR gate with inverted inputs.

A	B	$(A+B)'$	$A' \cdot B'$	Match?
0	0	1	1	YES
0	1	0	0	YES
1	0	0	0	YES
1	1	0	0	YES

[APPLICATION] De Morgan's theorem is used to convert NAND-NAND networks to SOP expressions and NOR-NOR networks to POS expressions — essential for K-Map simplification.

Q10. Define Flip-Flop and Its Types

■ *A flip-flop is a 1-bit memory cell — the building block of registers, counters, and sequential logic.*

A **flip-flop** is a bistable multivibrator — a sequential logic circuit that stores one bit of information and can hold either state (0 or 1) indefinitely until a clock edge triggers a state change.

Type	Inputs	Key Characteristic	Application
SR Flip-Flop	S, R	Set/Reset; forbidden state S=R=1	Basic memory cell
D Flip-Flop	D	Output follows D at clock edge	Data registers, shift registers
JK Flip-Flop	J, K	No invalid state; J=K=1 toggles	Counters, universal FF
T Flip-Flop	T	Toggles when T=1	Binary counters, frequency division

Q11. What is Static RAM and Dynamic RAM?

■ *SRAM is faster and costlier (used for cache); DRAM is denser and cheaper (used for main memory).*

Feature	SRAM	DRAM
Storage element	Flip-flop (6 transistors)	Capacitor + 1 transistor
Refresh needed	No	Yes — every ~64 ms
Speed	Very fast (~1 ns)	Slower (~10–50 ns)
Density	Low (less bits/chip)	High (GB per chip)
Power	Higher static power	Lower — power only when refreshing
Cost	Expensive	Cheap
Usage	L1/L2/L3 Cache	Main memory (RAM modules)

[DEEP-DIVE] DRAM capacitors leak charge over time. The memory controller must periodically read and rewrite (refresh) each row to prevent data loss. This refresh cycle causes DRAM to stall CPU access for ~5% of the time.

Q12. List Basic Computer Registers and Functions

■ *The 8 basic registers form the operational core of the CPU — together they implement the fetch-decode-execute cycle.*

Register	Full Name	Bits	Function
PC	Program Counter	12	Holds address of NEXT instruction to fetch
AR	Address Register	12	Holds memory address for current operation
DR	Data Register	16	Holds data read from / to be written to memory
AC	Accumulator	16	General-purpose register; holds ALU results
IR	Instruction Register	16	Holds currently executing instruction
TR	Temporary Register	16	Temporary storage during multi-step operations
INPR	Input Register	8	Holds character received from input device
OUTR	Output Register	8	Holds character to be sent to output device

Q13. What is IC? Write Its Types

■ *ICs revolutionised computing by shrinking entire circuits onto a single silicon chip.*

An **Integrated Circuit (IC)** is a miniaturised electronic circuit fabricated on a single semiconductor wafer (silicon chip). Thousands to billions of transistors, resistors, and capacitors are embedded on a chip area smaller than a fingernail.

Type	Full Form	Components per Chip	Example
SSI	Small Scale Integration	1–10 gates	74xx logic gates
MSI	Medium Scale Integration	10–100 gates	Multiplexers, decoders
LSI	Large Scale Integration	100–10,000 gates	8-bit microprocessors
VLSI	Very Large Scale Integration	10K–1M gates	32/64-bit CPUs, GPUs
ULSI	Ultra Large Scale Integration	>1 Million gates	Modern Intel/AMD CPUs

Q14. Define Hit Ratio

■ *Hit ratio quantifies how effectively the cache serves memory requests — higher is better.*

Hit Ratio (h) is the fraction of total memory accesses that are successfully served from cache memory (cache hit), as opposed to requiring a slower main memory access (cache miss).

$\text{Hit Ratio (h)} = \text{Number of Cache Hits} / \text{Total Memory Accesses}$

$\text{Miss Ratio} = 1 - h$

$\text{Avg Access Time (T}_{\text{avg}}) = h * T_{\text{cache}} + (1-h) * T_{\text{main}}$

Numerical Example:

Cache access time = 10 ns, Main memory = 100 ns, Hit ratio = 0.9

$T_{\text{avg}} = 0.9 \times 10 + 0.1 \times 100 = 9 + 10 = 19 \text{ ns}$

Q15. What is MIMD?

■ **MIMD is the most common parallel computing model — it underlies multi-core CPUs and distributed clusters.**

MIMD (Multiple Instruction, Multiple Data) is one of Flynn's four taxonomy categories for computer architectures. In MIMD systems, multiple processors simultaneously execute different instructions on different data streams, operating independently and asynchronously.

Flynn's Category	Instructions	Data	Example
SISD	Single	Single	Classic uniprocessor (old PC)
SIMD	Single	Multiple	GPU, vector processor, SSE instructions
MISD	Multiple	Single	Fault-tolerant systems (theoretical)
MIMD	Multiple	Multiple	Multi-core CPU, server clusters, cloud

[INSIGHT] Modern multi-core processors (Intel Core i9, AMD Ryzen) are MIMD systems. Each core independently executes its own instruction stream on its own data.

Q16. NOR Gate and NOT Gate — Truth Tables

■ **NOR is a universal gate — any logic function can be built using only NOR gates.**

NOT Gate (Inverter) — Single Input:

Input A	Output $Y = A'$
0	1
1	0

NOR Gate — Two Inputs ($Y = (A+B)'$):

A	B	A+B	$Y=(A+B)'$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

[KEY FACT] NOR and NAND are called universal gates because any Boolean function can be implemented using only NOR gates or only NAND gates. This is exploited in CMOS manufacturing.

Q17. Three Control Inputs for Registers

■ *Three control signals govern ALL data movement in and out of CPU registers.*

Control Input	Signal Name	Function	Active State
Load (LD)	Write Enable	Loads new data into the register from data bus	High (1)
Clear (CLR)	Reset	Asynchronously resets all register bits to 0	High (1)
Clock (CLK)	Synchronise	Triggers state change on rising or falling edge	Edge-triggered

Q18. Two Characteristics of Multiprocessor

■ *Multiprocessors share memory and OS, enabling true parallel execution of threads.*

- **Shared Memory:** All processors access a common, unified address space. Any processor can read or write any memory location. This enables low-latency inter-processor communication but requires synchronisation primitives (mutexes, semaphores) to prevent race conditions.
- **Parallel Processing:** Multiple processors execute different tasks (or parts of the same task) simultaneously, reducing total execution time. Amdahl's Law limits the speedup achievable based on the serial fraction of the program.
- **Scalability:** Additional processors can be added to improve throughput up to the limits of the memory bus bandwidth.
- **Single OS Instance:** One operating system manages all processors and presents them as a single coherent system to user applications.

SECTION B

Medium Answer Questions

5 Marks Each | Answer
ANY 4 of 6

Q19. K-Map Simplification (4-Variable)

■ *K-Map is the fastest method to minimise Boolean expressions — always group in powers of 2.*

Standard Question: $F(A,B,C,D) = \sum m(1,3,7,11,15) + \sum d(0,2,5)$

K-Map Rules (NEVER violate):

- Groups must be of size 1, 2, 4, 8, or 16 (powers of 2).
- Groups must be rectangular (including wrap-around edges).
- Don't-care terms (d) can be included in groups if they help form larger groups.
- Always form the LARGEST possible groups.
- Cover every minterm at least once; a minterm can be in multiple groups.

4-Variable K-Map Layout (Gray code ordering):

AB\CD	00	01	11	10
00	m0	m1	m3	m2
01	m4	m5	m7	m6
11	m12	m13	m15	m14
10	m8	m9	m11	m10

2025 Paper: Simplify $F(A,B,C,D) = \sum m(1,3,7,11,15) + \sum d(0,2,5)$

Group 1: m1, m3, m7, m15, m11 — column CD=01, 11 wrap → CD

Group 2: m0, m2 (don't care) + m1, m3 → A'B'

Simplified: $F = CD + A'B'$ (verify by substituting minterms)

[EXAM STRATEGY] In the exam, first fill the K-map grid, circle the groups with different coloured pens, then write the simplified expression. Marks are awarded for both the map AND the expression.

Q20. 4-to-1 Multiplexer

■ *A multiplexer is a data selector — it routes one of N inputs to a single output based on select lines.*

A 4-to-1 Multiplexer has 4 data inputs (I0–I3), 2 select lines (S1, S0), and 1 output (Y). Based on the 2-bit select code, one of the four input lines is connected to the output.

Boolean Expression:

$$Y = S1' \cdot S0' \cdot I0 + S1' \cdot S0 \cdot I1 + S1 \cdot S0' \cdot I2 + S1 \cdot S0 \cdot I3$$

S1	S0	Output Y
0	0	I0 (Input 0 selected)

S1	S0	Output Y
0	1	I1 (Input 1 selected)
1	0	I2 (Input 2 selected)
1	1	I3 (Input 3 selected)

Gate-Level Implementation:

- 4 AND gates (each 3-input: one select pair + data input)
- 2 NOT gates (to invert S1 and S0)
- 1 four-input OR gate (combines all AND outputs)
- Total: $4+2+1 = 7$ gates

[APPLICATION] Multiplexers implement any Boolean function. A 4-to-1 MUX with fixed select lines can implement any 2-variable function by wiring I0–I3 to the function's truth table column.

Q21. Half Adder and Full Adder

■ *Half adder adds 2 bits; full adder adds 3 bits — full adders chain together to add multi-bit numbers.*

HALF ADDER — Adds two 1-bit numbers (A, B):

$$\text{Sum (S)} = A \text{ XOR } B = A \oplus B$$

$$\text{Carry (C)} = A \text{ AND } B = A \cdot B$$

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

FULL ADDER — Adds three 1-bit numbers (A, B, Cin):

$$\text{Sum (S)} = A \text{ XOR } B \text{ XOR } \text{Cin}$$

$$\text{Carry (Cout)} = (A \cdot B) + (B \cdot \text{Cin}) + (A \cdot \text{Cin})$$

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0

A	B	Cin	Sum	Cout
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Implementation: Full Adder = 2 Half Adders + 1 OR gate

[KEY LINK] A 4-bit ripple-carry adder is built by cascading 4 full adders: Cout of stage N feeds Cin of stage N+1. The carry 'ripples' through each stage.

Q22. BUN and BSA Instructions

■ **BUN and BSA are the two fundamental branch instructions in the basic computer — they implement loops and subroutines.**

BUN — Branch Unconditionally:

BUN transfers program control to a specified address **unconditionally**, regardless of any condition flags. Equivalent to a GOTO statement.

Operation: $PC \leftarrow AR$

Effect: Next instruction fetched from address in AR

BSA — Branch and Save Address:

BSA implements **subroutine call** functionality. It saves the return address at the branch address, then jumps to branch_address+1 to execute the subroutine.

Operation: $M[AR] \leftarrow PC, PC \leftarrow AR + 1$

Step 1: Save current PC (return address) into memory at AR

Step 2: Jump to AR+1 (first instruction of subroutine)

Return: BUN to saved address restores execution flow

[EXAM TRAP] BSA stores the RETURN address at the branch address itself (memory[AR]), not at a stack. This is different from modern CALL instructions that use a stack. The subroutine starts at AR+1, not AR.

Q23. CISC vs RISC

■ **CISC trades instruction complexity for code density; RISC trades simplicity for execution speed.**

Feature	CISC	RISC
Full Form	Complex Instruction Set Computer	Reduced Instruction Set Computer
Instruction Count	Large (100–300+ instructions)	Small (50–100 instructions)
Instruction Length	Variable (1–15 bytes in x86)	Fixed (typically 32 bits)
Execution Cycles	Multiple cycles per instruction	1 cycle per instruction (ideal)

Feature	CISC	RISC
Memory Access	Any instruction can access memory	Only LOAD/STORE access memory
Registers	Fewer general-purpose registers	Many general-purpose registers (32+)
Pipeline	Difficult to pipeline	Highly pipelineable
Compiler	Simpler compiler design	Complex compiler (optimisation heavy)
Power	Higher power consumption	Lower power (used in mobile/ARM)
Examples	Intel x86, AMD64	ARM, MIPS, RISC-V, SPARC

Q24. Associative Memory

■ **Associative memory searches by content, not address — it's the hardware that makes cache lookup instantaneous.**

Also called **Content Addressable Memory (CAM)**, associative memory performs a parallel search across ALL stored words simultaneously using a search key (argument). Instead of specifying an address, you specify partial content and the memory returns all matching entries.

Structure:

- **Argument Register:** Holds the search key (what you're looking for).
- **Key Register (Mask):** Specifies which bits of the argument are relevant.
- **Match Register:** Output — one bit per word, set to 1 if that word matches.
- **Data Array:** Stores all words in parallel comparator cells.

Applications:

Application	How CAM is Used	Benefit
CPU Cache	Tag matching for cache hit detection	$O(1)$ lookup vs $O(n)$ search
TLB (Virtual Memory)	Virtual-to-physical address translation	Single-cycle page table lookup
Network Routing	IP address matching in routers	Line-rate packet forwarding
Database	Hash joins and key lookups	Parallel search across all rows

Q25. ILP — Instruction-Level Parallelism

■ **ILP is the degree to which instructions in a program can be executed simultaneously — limited by data and control dependencies.**

Instruction-Level Parallelism (ILP) refers to the potential for simultaneously executing multiple instructions from a sequential instruction stream by exploiting independence between instructions.

ILP Exploitation Techniques:

Technique	Mechanism	Hardware Required
Pipelining	Overlap fetch/decode/execute stages	Pipeline registers
Superscalar	Multiple execution units in parallel	Multiple ALUs/FPUs
Out-of-Order Execution	Execute independent instructions early	Reservation stations, ROB
Branch Prediction	Speculatively fetch predicted path	Branch target buffer
Register Renaming	Eliminate false (WAR/WAW) dependencies	Physical register file

Fundamental Limitations of ILP:

- **True Data Dependency (RAW):** Instruction B needs result from A — cannot reorder.
- **Anti-dependency (WAR):** B writes to register that A reads — partially solved by renaming.
- **Output Dependency (WAW):** Two writes to same register — solved by renaming.
- **Control Dependencies:** Branch outcomes unknown until executed.
- **Memory Aliases:** Unknown if two memory addresses are the same.

[AMDAHL'S LAW] Speedup = $1 / (S + P/N)$ where S=serial fraction, P=parallel fraction, N=processors. Even with infinite processors, speedup is limited by the serial portion.

Q26. Types of Binary Codes

■ **Binary codes are systematic encodings that map decimal or character data into binary patterns with specific properties.**

Code	Full Name	Encoding Rule	Example
BCD	Binary Coded Decimal	Each decimal digit → 4 binary bits	25 → 0010 0101
Gray	Gray Code / Reflected Binary	Adjacent values differ by 1 bit only	3 → 0010, 4 → 0110
ASCII	American Standard Code for Info Interchange	7-bit code for 128 characters	'A' → 100 0001 (65)
Excess-3	XS-3 Code	BCD + 3 for each digit	5 → 0101+0011=1000
2421 Code	Weighted BCD variant	Weights: 2,4,2,1 — self-complementing	5 → 1011

[INSIGHT] Gray code is used in rotary encoders and ADCs because mechanical systems can momentarily show ambiguous states during transitions — Gray code ensures only 1 bit is ambiguous at any time.

Q27. Arithmetic Micro-operations

■ **Arithmetic micro-operations are the numerical building blocks of all computation — they directly map to ALU hardware.**

Operation	RTL Notation	Description	Hardware
Addition	$R3 \leftarrow R1 + R2$	Adds contents of R1 and R2, result in R3	Adder circuit
Subtraction	$R3 \leftarrow R1 - R2$	R1 minus R2 (implemented as $R1 + 2's \text{ comp } R2$)	Adder + NOT + 1

Operation	RTL Notation	Description	Hardware
Increment	$R1 \leftarrow R1 + 1$	Adds 1 to register contents	Adder with Cin=1
Decrement	$R1 \leftarrow R1 - 1$	Subtracts 1 from register	Adder, add (-1)
1's Complement	$R1 \leftarrow R1'$	Inverts all bits of R1	NOT gate per bit
2's Complement	$R1 \leftarrow R1' + 1$	Negation; 1's comp + 1	NOT + Increment
Multiply	$R3 \leftarrow R1 \times R2$	Partial-product accumulation	Array multiplier

Q28. 3-to-8 Line Decoder (Binary to Octal)

■ A decoder activates exactly ONE of 2^n outputs based on n input lines — the inverse of an encoder.

A **3-to-8 line decoder** has 3 input lines (A_2, A_1, A_0) and 8 output lines (D_0 – D_7). Exactly one output is HIGH at any time, corresponding to the decimal equivalent of the binary input.

$D_0 = A_2' . A_1' . A_0'$ (when input = 000)

$D_1 = A_2' . A_1' . A_0$ (when input = 001)

$D_2 = A_2' . A_1 . A_0'$ (when input = 010)

$D_7 = A_2 . A_1 . A_0$ (when input = 111)

A0	D0	D1	D2	D3	D4	D5
0	1	0	0	0	0	0
1	0	1	0	0	0	0
0	0	0	1	0	0	0
1	0	0	0	1	0	0
0	0	0	0	0	1	0
1	0	0	0	0	0	1
0	0	0	0	0	0	0
1	0	0	0	0	0	0

SECTION C

Long Answer Questions

8 Marks Each | Answer
ANY 4 of 6

Q29. SR Flip-Flop — Full Detailed Answer

■ *The SR flip-flop is the foundational 1-bit memory cell — all other flip-flops derive from it.*

An **SR (Set-Reset) Flip-Flop** is a bistable sequential circuit that can store one bit of binary information. It has two inputs — S (Set) and R (Reset) — and two outputs — Q and Q' (complement).

Construction Method 1 — Using NOR Gates:

- Two cross-coupled NOR gates where output of each feeds back as input to the other.
- Gate 1: $Q = (R + Q')'$ — NOR of R and Q'
- Gate 2: $Q' = (S + Q)'$ — NOR of S and Q

Construction Method 2 — Using NAND Gates:

- Active-LOW version: inputs are S' and R' (inverted).
- NAND SR: S'=0,R'=1 → Set; S'=1,R'=0 → Reset; S'=R'=1 → Hold.

Truth Table and State Transitions:

S	R	Q (next)	Q' (next)	State
0	0	Q	Q'	No Change (Hold)
0	1	0	1	Reset (Q=0)
1	0	1	0	Set (Q=1)
1	1	X	X	FORBIDDEN (undefined)

Clocked SR Flip-Flop:

A clock input (CLK) is ANDed with S and R inputs. The flip-flop changes state only on the active clock edge, enabling synchronous circuit design.

$S_{internal} = S \text{ AND CLK}$

$R_{internal} = R \text{ AND CLK}$

Characteristic Equation:

$Q(next) = S + R'.Q$ (condition: $S.R = 0$)

Applications of SR Flip-Flop:

Application	How SR FF is Used	S signal	R signal
Debounce switch	Latch first state change	Switch press	Switch release
Memory cell	Store 1-bit data	Write 1	Write 0
Alarm circuit	Hold alarm state	Trigger event	Reset button
Control register bit	Flag set/clear	Set condition	Clear condition

[FORBIDDEN STATE] When $S=R=1$ in NOR-based SR: both Q and Q' simultaneously become 0, violating the complementary output rule. When the inputs return to 0,0, the next state is unpredictable (race condition). This is why JK FF was invented — it eliminates this forbidden state.

Q30. DMA Controller — Working and Block Diagram

■ **DMA offloads bulk data transfer from the CPU to a dedicated controller, freeing the CPU for computation during I/O.**

Direct Memory Access (DMA) is a hardware mechanism that allows I/O devices to transfer data directly to/from main memory without requiring the CPU to execute each data transfer instruction individually. The DMA controller acts as a secondary bus master.

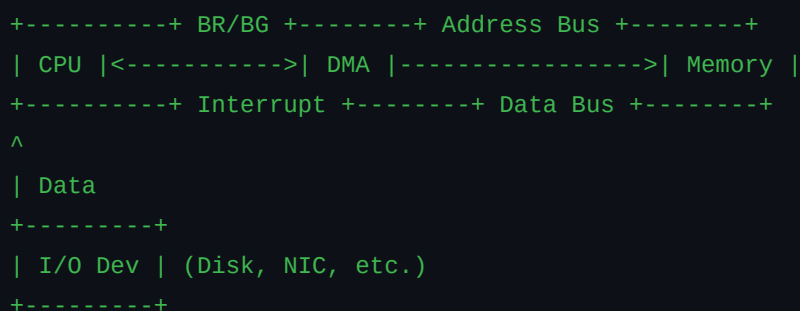
DMA Controller Internal Registers:

Register	Size	Content / Purpose
Address Register	16-bit	Holds starting memory address for the transfer
Word Count Register	16-bit	Number of words/bytes remaining to transfer
Data Register	16-bit	Temporary buffer between device and memory
Status/Control Reg	8-bit	Interrupt enable, read/write mode, transfer complete flag

DMA Transfer Process — Step by Step:

- **Step 1 — Initialisation:** CPU programs the DMA controller with starting memory address, transfer count, and direction (read/write). CPU then continues executing other tasks.
- **Step 2 — Bus Request:** When device is ready (e.g., disk sector loaded), DMA asserts BR (Bus Request) signal to the CPU.
- **Step 3 — Bus Grant:** CPU completes current memory cycle, asserts BG (Bus Grant), and tri-states its bus drivers (releases the bus).
- **Step 4 — Data Transfer:** DMA takes control of address and data bus. Places memory address on address bus, reads data from I/O device into data register, writes to memory. Increments address register, decrements word count.
- **Step 5 — Repeat:** DMA transfers one word per bus cycle. Steps 2–4 repeat until word count reaches zero.
- **Step 6 — Interrupt:** DMA asserts interrupt line (IEN). CPU receives interrupt, resumes data processing using the transferred data.

DMA Block Diagram Components:



DMA Transfer Modes:

Mode	Mechanism	CPU Impact	Use Case
Burst Mode	DMA holds bus for entire block transfer	CPU blocked for duration	Large block transfers
Cycle Stealing	DMA steals 1 bus cycle at a time	Minimal CPU slowdown	Real-time systems
Transparent Mode	DMA transfers only when CPU doesn't need bus	Zero CPU impact	Low-priority background transfers

[WHY DMA MATTERS] Without DMA, a CPU transferring a 512-byte disk sector at 1 byte/instruction would spend 512 I/O instructions. With DMA, the CPU issues ONE setup command and the DMA handles all 512 transfers autonomously.

Q31. 3×8 Line Decoder — Full Answer

■ *A decoder is a demultiplexer of control signals — it activates exactly one output line for every unique input combination.*

A **3-to-8 line decoder** converts a 3-bit binary address (A2, A1, A0) into one of 8 active output lines (D0–D7). It has one **Enable (E)** input — when E=0, all outputs are LOW regardless of inputs.

Boolean Output Equations:

$D0 = E \cdot A2' \cdot A1' \cdot A0'$

$D1 = E \cdot A2' \cdot A1' \cdot A0$

$D2 = E \cdot A2' \cdot A1 \cdot A0'$

$D3 = E \cdot A2' \cdot A1 \cdot A0$

$D4 = E \cdot A2 \cdot A1' \cdot A0'$

$D5 = E \cdot A2 \cdot A1' \cdot A0$

$D6 = E \cdot A2 \cdot A1 \cdot A0'$

$D7 = E \cdot A2 \cdot A1 \cdot A0$

Complete Truth Table (Enable E=1):

A0	D0	D1	D2	D3	D4	D5
0	1	0	0	0	0	0
1	0	1	0	0	0	0
0	0	0	1	0	0	0
1	0	0	0	1	0	0
0	0	0	0	0	1	0
1	0	0	0	0	0	1
0	0	0	0	0	0	0
1	0	0	0	0	0	0

Gate-Level Implementation:

- 3 NOT gates (to generate A2', A1', A0')
- 8 three-input AND gates (one per output, includes E input)
- Total: 11 gates

Expanding Decoders:

From	To	Method	Gates Needed
2-to-4	3-to-8	Two 2-to-4 decoders + enable logic	2 decoders + 1 NOT
3-to-8	4-to-16	Two 3-to-8 decoders + 1 NOT gate	2 decoders + 1 NOT
3-to-8	5-to-32	Four 3-to-8 + one 2-to-4	Complex tree

Applications:

- **Memory address decoding:** Select one of 8 memory chips based on upper address bits.
- **Instruction decoding:** Activate one of 8 micro-operation sequences.
- **Demultiplexer:** Route data to one of 8 destinations.

Q32. Instruction Cycle — All Phases and Micro-operations

■ *The instruction cycle is the fundamental heartbeat of every CPU — it repeats billions of times per second.*

The **Instruction Cycle** (also called the Fetch-Decode-Execute cycle) is the complete process the CPU performs for every single machine instruction. It consists of multiple sub-cycles, each composed of register transfer micro-operations.

Phase 1 — FETCH (Get instruction from memory):

```
T0: AR ← PC (load address register from PC)
T1: IR ← M[AR], PC ← PC+1 (read memory into IR, advance PC)
```

Phase 2 — DECODE (Determine instruction type):

```
T2: AR ← IR(0-11), D ← Decode(IR(12-14))
IR[12-14] = opcode bits, IR[0-11] = address field
D0-D7 decoder outputs determine instruction type
```

Phase 3 — INDIRECT ADDRESS (if I-bit = 1):

```
T3: if I=1: AR ← M[AR] (fetch effective address)
```

Phase 4 — EXECUTE (Perform the operation):

The execution sub-cycle depends on the decoded instruction type:

Instruction Type	Micro-operations	Description
AND (D0)	T4: DR ← M[AR]; T5: AC ← AC ∧ DR	AND memory with AC
ADD (D1)	T4: DR ← M[AR]; T5: AC ← AC + DR, E ← Cout	Add memory to AC
LDA (D2)	T4: DR ← M[AR]; T5: AC ← DR	Load memory into AC
STA (D3)	T4: M[AR] ← AC	Store AC into memory
BUN (D4)	T4: PC ← AR	Unconditional branch
BSA (D5)	T4: M[AR] ← PC, AR ← AR + 1; T5: PC ← AR	Branch and save address
ISZ (D6)	T4: DR ← M[AR]; T5: DR ← DR + 1; T6: M[AR] ← DR, if DR=0: PC ← PC + 1	Increment and skip

Instruction Cycle Flowchart (text representation):

```

START → [FETCH: AR←PC, IR←M[AR], PC←PC+1]
→ [DECODE: AR←IR(addr), D←decode(IR(op)) ]
→ {I=1?} YES→[AR←M[AR]] NO→skip
→ {D0-D6?} → [EXECUTE appropriate micro-ops]
→ {D7, I/O?} → [Input/Output micro-ops]
→ LOOP back to FETCH

```

[TIMING] Each micro-operation takes exactly one clock cycle (T period). A full instruction may take T₀ through T₅ (6 clock cycles) or T₀–T₃ for simple instructions. The control unit uses a timing ring counter to generate T₀, T₁, ..., T_n signals.

Q33. Addressing Modes — Five Modes with Examples

■ **Addressing modes define how the CPU computes the Effective Address (EA) of the operand — they are the flexibility mechanism of ISA design.**

Mode	Effective Address	Example	Advantage
Immediate	Operand IS the data	ADD #5 (adds 5 directly)	No memory access needed
Direct (Absolute)	EA = Address field	ADD 500 (adds M[500])	Simple, 1 memory access
Indirect	EA = M[Address field]	ADD @500 (EA = M[500])	Pointer-style access
Register Direct	EA = Register	ADD R1 (adds value in R1)	Fastest — no memory
Register Indirect	EA = M[Register]	ADD (R1) (EA = M[R1])	Array/pointer traversal
Displacement/Based	EA = Base_Reg + Offset	ADD 10(R1)	Array indexing, stack frames
Relative	EA = PC + Offset	JUMP +5 (jump 5 ahead)	Position-independent code

Numerical Example (Indirect Addressing):

```

Instruction: LOAD INDIRECT 500
M[500] = 800 (500 holds pointer to actual data)
M[800] = 42 (actual data)
Result: AC ← 42 (two memory accesses total)

```

[EXAM TRAP] Immediate mode: the operand IS the constant value. Direct mode: the address field IS the memory address. Indirect mode: the address field points to the location that CONTAINS the actual address. Students confuse direct and indirect.

Q34. Multithreaded Architecture

■ **Multithreading hides memory latency by switching to another thread while one thread waits for data — it keeps the CPU pipeline full.**

Multithreading is a hardware technique that allows a single CPU core to maintain multiple thread contexts simultaneously, switching between them to hide latency (especially memory access latency of 100–300 cycles).

Hardware Required:

- Multiple Program Counters (one per thread context)
- Multiple Register Files (or partitioned register file)
- Multiple Stack Pointers and status registers
- Thread scheduler in the control unit

Type	Switch Mechanism	Granularity	Latency Hidden	Example
Fine-Grained (FMT)	Switch every clock cycle, round-robin	1 instruction	Short latencies (cache miss ~5 cycles)	Sun UltraSPARC T1
Coarse-Grained (CMT)	Switch only on long-latency stall (cache miss)	Multiple instructions	Long latencies (DRAM ~200 cycles)	IBM AS/400
Simultaneous (SMT)	Multiple threads execute SAME cycle on different execution units	1 cycle, parallel	All types	Intel Hyper-Threading

Fine-Grained vs Coarse-Grained — Key Differences:

Aspect	Fine-Grained	Coarse-Grained
Switch trigger	Every clock cycle	Only on cache miss/stall
Overhead	Zero (always switching)	Low (rare switches)
Single-thread performance	Poor (diluted per thread)	Good (runs until stall)
Throughput	High for many threads	Moderate
Pipeline stages needed	N threads × pipeline depth	Smaller buffer requirement

[SMT vs MULTICORE] SMT (Hyper-Threading) makes 1 physical core appear as 2 logical cores. A true dual-core has 2 complete physical execution units. SMT improves throughput by ~25-30%; dual-core doubles throughput for parallel workloads.

Q35. Shift Registers with Parallel Load

■ *A shift register with parallel load is the hardware foundation of serial-to-parallel data conversion used in UART, SPI, and I2C interfaces.*

A **shift register** is a cascade of D flip-flops where the output of each feeds the input of the next. It shifts data left or right by one bit position per clock cycle.

Type	Full Name	Data In	Data Out	Application
SISO	Serial In, Serial Out	1 bit/cycle	1 bit/cycle	Data delay lines, LFSR
SIPO	Serial In, Parallel Out	1 bit/cycle	n bits at once	Serial-to-parallel conversion (UART RX)
PISO	Parallel In, Serial Out	n bits at once	1 bit/cycle	Parallel-to-serial conversion (UART TX)
PIPO	Parallel In, Parallel Out	n bits at once	n bits at once	CPU registers, buffer registers

4-bit PISO Shift Register with Parallel Load — Operation:

- **Load Mode (Load/Shift = 1):** All flip-flops simultaneously load their parallel input D0–D3. The 4-bit word is captured in one clock cycle.
- **Shift Mode (Load/Shift = 0):** Each clock cycle, $FF0 \rightarrow FF1 \rightarrow FF2 \rightarrow FF3$, MSB shifts out serially.
- **Control Logic per FF:** $\text{Output} = (\text{Load} \cdot \text{Di}) + (\text{Shift} \cdot \text{Q}_{\text{prev}})$

Cascaded Shift Register Micro-operations:

```

shl R → Ri ← R(i-1), R0 ← 0 (shift left, 0 fills LSB)
shr R → Ri ← R(i+1), Rn ← 0 (shift right, 0 fills MSB)
cil R → Ri ← R(i-1), R0 ← Rn (circular left, wrap MSB)
cir R → Ri ← R(i+1), Rn ← R0 (circular right, wrap LSB)

```

[INSIGHT] Left-shifting by 1 bit is equivalent to multiplying by 2. Right-shifting by 1 bit is equivalent to dividing by 2 (integer division). This is how early CPUs implemented fast multiplication.

Q36. Register-Reference Instructions

■ **Register-reference instructions operate directly on the AC register — they need NO memory access and execute in a single clock cycle.**

Register-reference instructions are identified by opcode **D7=1, I=0** in the basic computer. They use bits 0–11 of the instruction register to specify the operation (one bit per operation).

Mnemonic	Bit	RTL Operation	Description
CLA	B11	$AC \leftarrow 0$	Clear the Accumulator
CLE	B10	$E \leftarrow 0$	Clear the E (carry) flag
CMA	B9	$AC \leftarrow AC'$	Complement the Accumulator (1's complement)
CME	B8	$E \leftarrow E'$	Complement the E bit
CIR	B7	$AC \leftarrow \text{shr } AC, AC[15] \leftarrow E, E \leftarrow AC[0]$	Circular shift right (AC and E)
CIL	B6	$AC \leftarrow \text{shl } AC, AC[0] \leftarrow E, E \leftarrow AC[15]$	Circular shift left (AC and E)
INC	B5	$AC \leftarrow AC + 1$	Increment the Accumulator
SPA	B4	if $AC[15]=0$: $PC \leftarrow PC+1$	Skip if AC positive (MSB=0)
SNA	B3	if $AC[15]=1$: $PC \leftarrow PC+1$	Skip if AC negative (MSB=1)
SZA	B2	if $AC=0$: $PC \leftarrow PC+1$	Skip if AC is Zero
SZE	B1	if $E=0$: $PC \leftarrow PC+1$	Skip if E is Zero
HLT	B0	$S \leftarrow 0$	Halt the computer (stop clock)

[DEEP-DIVE] The skip instructions (SPA, SNA, SZA, SZE) implement conditional branching without a dedicated branch opcode. They skip the NEXT instruction (usually a BUN). Combined: $\text{SPA} \rightarrow \text{BUN label} \rightarrow$ means 'if positive, jump to label'.

Q37. Cross-bar Switch and Multi-stage Switching Network

■ **Crossbar switches provide full non-blocking connectivity; multi-stage networks reduce hardware cost at the price of potential blocking.**

In multiprocessor systems, processors must communicate with memory modules. The **interconnection network** determines the bandwidth and latency of this communication.

Crossbar Switch:

An $N \times M$ crossbar switch connects N processors to M memory modules using $N \times M$ switching elements (intersection points). Any processor can be connected to any memory module simultaneously as long as no two processors access the same module.

- **Non-blocking:** N processors can access N different memories simultaneously.
- **Hardware cost:** $O(N^2)$ switching elements — expensive for large N .
- **Complexity:** $O(N^2)$ — feasible up to ~ 16 processors.

4×4 Crossbar (P=Processors, M=Memory):

```

M0 M1 M2 M3
P0 [ X ] [ ] [ ] [ ]
P1 [ ] [ X ] [ ] [ ]
P2 [ ] [ ] [ X ] [ ]
P3 [ ] [ ] [ ] [ X ]
X = active connection

```

Multi-stage Interconnection Network (MIN):

A MIN uses multiple stages of smaller switching elements (typically 2×2) to connect N inputs to N outputs with fewer total switches than a crossbar.

Network	Stages	Switches	Blocking?	Examples
Crossbar	1	N^2	No (non-blocking)	Small shared-memory MPs
Omega Network	$\log_2 N$	$N/2 \times \log_2 N$	Yes (blocking)	BBN Butterfly
Banyan Network	$\log_2 N$	$N/2 \times \log_2 N$	Yes (blocking)	Research systems
Benes Network	$2\log_2 N - 1$	$N(2\log_2 N - 1)/2$	No (rearrangeable)	Telephone switching

[BLOCKING] A network is 'blocking' if there exists a valid routing request that CANNOT be satisfied because intermediate switch stages are already occupied. Omega network is blocking — two different source-destination pairs may require the same intermediate switch.

Q38. Types of CPU Organization

■ **CPU organisation defines how registers, ALU, and memory communicate — the bus structure determines instruction throughput.**

Organization	Bus Structure	Operations	Instruction Speed	Hardware Cost
Single Accumulator	1 data path through AC	AC-based only	Slow (many instructions)	Minimal

Organization	Bus Structure	Operations	Instruction Speed	Hardware Cost
General Register	Multiple registers, shared bus	Flexible register ops	Medium	Moderate
Single Bus	1 common bus for all	1 transfer/cycle	Slow (bus bottleneck)	Low
Two Bus	2 parallel buses	2 transfers/cycle	Faster	Medium
Three Bus	3 buses: A, B, Result	ALU: $R_d \leftarrow R_{s1} \text{ op } R_{s2}$	Fastest (1 cycle)	High

Three-Bus Architecture — Single Instruction Example:

ADD R3, R1, R2:

Bus A carries R1 value to ALU input A

Bus B carries R2 value to ALU input B

ALU computes $R1+R2$ in one cycle

Result bus carries sum to R3

Single-bus equivalent requires 3 separate micro-operations (3 cycles)

Q39. Full Adder — Detailed Design

■ *Full adder is the core building block of all arithmetic operations in digital computers.*

A **Full Adder** accepts three 1-bit inputs (A, B, Cin) and produces a 2-bit output: Sum (S) and Carry-out (Cout). Unlike a half adder, it handles the carry-in from a previous stage, enabling cascaded multi-bit addition.

Boolean Expressions:

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = (A \cdot B) + (B \cdot C_{in}) + (A \cdot C_{in}) \text{ [majority function]}$$

Full Truth Table:

A	B	Cin	Sum (S)	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Implementation using two Half Adders:

$$HA1: S1 = A \oplus B, C1 = A \cdot B$$

$$HA2: S = S1 \oplus C_{in}, C2 = S1 \cdot C_{in}$$

$$C_{out} = C1 + C2$$

Total gates: 2 XOR + 2 AND + 1 OR = 5 gates

Q40. De Morgan's Theorem — State and Prove

■ De Morgan's theorem is the bridge between NOR/NAND gates and AND/OR logic — it enables universal gate implementations.

FIRST LAW: $(A + B)' = A' \cdot B'$

The complement of a logical OR equals the logical AND of the complements.

Proof by Perfect Induction (Truth Table):

A	B	A+B	(A+B)'	A'	B'	A'·B'	Equation
0	0	0	1	1	1	1	YES
0	1	1	0	1	0	0	YES
1	0	1	0	0	1	0	YES
1	1	1	0	0	0	0	YES

SECOND LAW: $(A \cdot B)' = A' + B'$

The complement of a logical AND equals the logical OR of the complements.

A	B	A·B	(A·B)'	A'	B'	A'+B'	Equation
0	0	0	1	1	1	1	YES
0	1	0	1	1	0	1	YES
1	0	0	1	0	1	1	YES
1	1	1	0	0	0	0	YES

Gate Equivalences (Direct Application):

Gate	De Morgan Equivalent	Diagram Representation
NOR(A,B)	AND(A', B')	OR → bubble on output = AND with bubbles on inputs
NAND(A,B)	OR(A', B')	AND → bubble on output = OR with bubbles on inputs
NOT(A+B)	A'·B'	NOR = AND-with-inverted-inputs
NOT(A·B)	A'+B'	NAND = OR-with-inverted-inputs

Q41. CISC vs RISC — Extended 8-Mark Answer

■ The CISC-RISC debate shaped modern processor design — today's x86 CPUs decode CISC instructions into RISC-like micro-ops internally.

Historical Context:

CISC emerged in the 1970s when memory was expensive — complex instructions reduced program size. RISC emerged in the 1980s (Patterson/Hennessy at Berkeley/Stanford) based on the observation that 80% of execution time is spent on 20% of instructions — simplifying those yielded massive gains.

Design Dimension	CISC Architecture	RISC Architecture
Instruction philosophy	One instruction does more work	One instruction does one operation
Code density	High (fewer instructions/program)	Low (more instructions/program)
Instruction format	Variable length (1–15 bytes)	Fixed length (4 bytes)
Execution model	Microcode-based multi-cycle	Hardwired single-cycle
Register count	Few (8 in original x86)	Many (32 in MIPS/ARM)
Memory operands	Allowed in arithmetic ops	LOAD/STORE only
Pipeline efficiency	Low — unequal instruction times	High — equal instruction times
Compiler complexity	Low — rich instruction set	High — compiler must schedule
Cache benefit	Higher miss penalty (complex decode)	Cache-friendly fixed fetch
Power profile	High power (mobile-unfriendly)	Low power (mobile-dominant)
Modern examples	Intel Core, AMD Ryzen (x86-64)	ARM Cortex, Apple M-series, RISC-V

[MODERN REALITY] Modern x86 CPUs (Intel Sandy Bridge+) internally decode CISC instructions into fixed-width RISC micro-ops before execution. The front-end is CISC-compatible; the back-end is a RISC engine. This gives code compatibility while achieving RISC execution efficiency.

Q42. I/O Interface Unit

■ *The I/O interface unit bridges the semantic and electrical gap between the CPU's fast parallel bus and slow, diverse peripheral devices.*

An **I/O Interface Unit** (also called I/O Controller or Device Controller) is the hardware module that manages communication between the CPU/memory system and an external I/O device. It handles electrical signal conversion, data format conversion, timing synchronisation, and status monitoring.

Internal Structure of an I/O Interface:

Component	Function	Size
Data Register	Buffers data being transferred (CPU↔Device)	8–32 bits
Status Register	Holds device state: BUSY, READY, ERROR, DONE	8 bits
Control Register	Accepts commands from CPU (START, STOP, MODE)	8 bits
Address Decoder	Decodes I/O port address to select this device	Combinational logic
Data Buffer	Matches device speed to bus speed (FIFO)	Variable
Interrupt Logic	Asserts IRQ when transfer complete	1 flip-flop + logic
Bus Interface	Tri-state drivers for data/address bus	Bus width

I/O Transfer Methods Comparison:

Method	CPU Involvement	Mechanism	Performance
Programmed I/O	100% (busy-wait)	CPU polls status register in loop	Poor — CPU wasted
Interrupt-driven I/O	Only on interrupt	Device interrupts CPU when ready	Good — CPU free between transfers
DMA	Only setup+completion	DMA controller manages transfer	Best — CPU fully free during transfer

[INSIGHT] Modern systems use a layered approach: small/frequent data via interrupts; bulk transfers via DMA. A USB keyboard uses interrupts; a USB hard drive uses DMA. The OS device driver selects the appropriate method.

Q43. Multiprocessor vs Multicomputer — Detailed Comparison

■ **Multiprocessors share memory (tight coupling); multicomputers share nothing (loose coupling) — the difference determines programming model and scalability.**

Dimension	Multiprocessor	Multicomputer
Memory	Shared — all CPUs see same address space	Private — each node has own memory
Communication	Read/write shared variables	Message Passing (MPI, send/receive)
Coupling	Tightly coupled	Loosely coupled
Interconnect	Shared bus or crossbar switch	Network (Ethernet, InfiniBand)
OS	Single OS instance manages all	Each node runs own OS
Scalability	Limited (~64 processors) — bus contention	Highly scalable (thousands of nodes)
Latency	Low — shared memory ~100 ns	High — network ~1–100 μ s
Synchronisation	Hardware primitives (test-and-set)	Explicit in messages
Cost	Expensive (specialised hardware)	Cheap commodity hardware
Fault tolerance	Single point of failure (shared bus)	High — node failure isolated
Examples	Intel SMP, Sun Fire servers	Beowulf cluster, Google data centre, HPC grids

Cache Coherence Problem (Multiprocessors only):

When multiple processors have private caches containing copies of the same memory location, a write by one processor makes other caches stale. Solutions include:

- **Write-Invalidate (MESI protocol):** Writing processor marks all other copies invalid. Used by Intel x86.
- **Write-Update (Dragon protocol):** Writing processor updates all copies simultaneously. Higher bus traffic but faster subsequent reads.

[NUMA vs UMA] Uniform Memory Access (UMA): All processors have equal latency to all memory — simple but doesn't scale. Non-Uniform Memory Access (NUMA): Processors access local memory faster than remote — scales to 1000+ processors but programming is harder.

QUICK REFERENCE — FORMULAS AND KEY FACTS

Topic	Key Formula / Fact	Notes
Hit Ratio	$h = \text{Hits} / (\text{Hits} + \text{Misses})$	Range: 0 to 1; good cache: >0.95
Avg Access Time	$T = h \cdot T_c + (1-h) \cdot T_m$	T_c = cache time, T_m = main memory time
Full Adder Sum	$S = A \oplus B \oplus C_{in}$	XOR of all three inputs
Full Adder Carry	$C_{out} = AB + BC_{in} + AC_{in}$	Majority function
De Morgan 1	$(A+B)' = A' \cdot B'$	NOR = AND with inverted inputs
De Morgan 2	$(A \cdot B)' = A' + B'$	NAND = OR with inverted inputs
4:1 MUX output	$Y = S_1'S_0'I_0 + S_1'S_0'I_1 + S_1S_0'I_2 + S_1S_0'I_3$	2 select lines
Gray Code MSB	$G[n] = B[n]$	MSB copied directly
Gray Code rest	$G[i] = B[i] \oplus B[i+1]$	XOR with next higher bit
BUN operation	$PC \leftarrow AR$	Unconditional branch
BSA operation	$M[AR] \leftarrow PC, PC \leftarrow AR+1$	Save return address, jump
Decoder outputs	$D[i] = \text{AND of address bits/complements}$	Exactly 1 active at a time
SR FF char. eq.	$Q(\text{next}) = S + R' \cdot Q$	Condition: $S \cdot R = 0$
Cache levels	$L1 < L2 < L3 < \text{RAM}$	Speed: L1 fastest; Size: RAM largest
RISC vs CISC	RISC: fixed width, 1-cycle; CISC: variable, multi-cycle	
DMA modes	Burst / Cycle Stealing / Transparent	Cycle stealing most common

EXAM STRATEGY — SECTION-WISE

Section	Strategy	Time Budget	Marks
Section A (2M)	Answer all 4 — definitions + 1 sentence explanation. No elaboration needed.	10 min	8
Section B (5M)	Choose 4 easiest. Tables/truth tables get full marks fast. K-Map always attempt.	35 min	20
Section C (8M)	Prioritise: SR FF, DMA, Instruction Cycle, Decoder. Draw neat diagrams. Write equations.	75 min	32

HIGH-PROBABILITY QUESTIONS FOR 2025-26

Rank	Topic	Section	Appeared In
#1	SR Flip-Flop	C	Both 2024 and 2025 papers
#2	DMA Controller	C	Both 2024 and 2025 papers
#3	Instruction Cycle + Flowchart	C	Both papers + question bank
#4	3×8 Line Decoder	C/B	Both papers + question bank
#5	K-Map Simplification	B	Every paper
#6	Addressing Modes	C	Question bank priority
#7	CISC vs RISC	B/C	Both papers
#8	Full Adder	B/C	Both papers
#9	Multithreaded Architecture	C	Both papers
#10	Register-reference instructions	C	2025 paper + question bank